

UNIDAD 5

Lectura y escritura de información (RA5)

E/S por consola · E/S con ficheros · Excepciones

**Objetivos de aprendizaje de esta unidad**

Leer datos por consola con **Scanner** y mostrar resultados formateados.
Usar **printf()** y los especificadores de formato para presentar datos.
Leer y escribir **ficheros de texto** con `BufferedReader` / `BufferedWriter`.
Gestionar **excepciones de E/S** con `try-catch` y `try-with-resources`.
Cargar datos de un fichero CSV y crear **objetos** a partir de ellos.

Mapa de contenidos

Bloque 5.1 — E/S por consola	Bloque 5.2 — E/S con ficheros
• <code>System.out.println</code> / <code>print</code> • <code>Scanner</code> (<code>nextLine</code>, <code>nextInt</code>...) • <code>printf()</code> y especificadores • Problema del buffer	• Streams (byte vs. carácter) • <code>FileReader</code> / <code>BufferedReader</code> • <code>FileWriter</code> / <code>BufferedWriter</code> • <code>Try-with-resources</code> • Objetos y CSV

Bloque 5.1 · Conceptos básicos de E/S por consola

1. Introducción a la E/S en Java

La **entrada/salida (E/S)** permite a los programas Java comunicarse con el usuario a través de la consola: mostrar resultados por pantalla y leer datos desde el teclado.



Funciones principales de E/S en Java

System.out.println() / **System.out.print()** — mostrar información.

System.out.printf() — mostrar información con formato preciso.

Clase **Scanner** — leer datos desde el teclado (o desde un fichero).

2. Salida de datos por pantalla

Java ofrece tres métodos principales para imprimir en la consola:

JAVA — Métodos de salida básicos

```
System.out.println("Hola, mundo!"); // Salta de línea al final
System.out.print("Sin salto");      // No salta de línea
System.out.println(" - continúa."); // Sigue en la misma línea
System.out.printf("Valor: %d\n", 42); // Formato + salto de línea
```

Método	Comportamiento	Ejemplo
<code>println()</code>	Imprime y salta de línea	<code>System.out.println("Hola");</code>
<code>print()</code>	Imprime sin saltar	<code>System.out.print("Hola");</code>
<code>printf()</code>	Imprime con formato	<code>System.out.printf("%s\n", "Hi");</code>

3. Entrada de datos con Scanner

La clase **Scanner** (paquete **java.util**) permite leer datos introducidos por el usuario desde teclado.



Scanner — Definición

Clase que lee la entrada de un flujo (teclado, fichero...) y la convierte en el tipo de dato necesario: **String**, **int**, **double**, etc.

JAVA — Ejemplo completo con Scanner

```
import java.util.Scanner;

public class EntradaDatos {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Introduce tu nombre: ");
        String nombre = teclado.nextLine();

        System.out.print("Introduce tu edad: ");
        int edad = Integer.parseInt(teclado.nextLine());
    }
}
```

```

        System.out.println("Hola, " + nombre + " . Tienes " + edad + " años.");
        teclado.close();
    }
}

```

Método Scanner	Lee	Ejemplo
<code>nextLine()</code>	Una línea completa (String)	<code>String s = sc.nextLine();</code>
<code>nextInt()</code>	Un número entero	<code>int n = sc.nextInt();</code>
<code>nextDouble()</code>	Un número decimal	<code>double d = sc.nextDouble();</code>
<code>nextFloat()</code>	Un float	<code>float f = sc.nextFloat();</code>
<code>next()</code>	Una palabra (hasta espacio)	<code>String p = sc.next();</code>

3.1 El problema del buffer

Uno de los problemas más frecuentes con Scanner es que **`nextLine()` puede saltarse** cuando se usa justo después de `nextInt()` o `nextDouble()`.

⚠ Problema del buffer de entrada

Cuando **`nextInt()`** lee un número, deja el carácter '\n' en el buffer. El **`nextLine()`** siguiente lee ese '\n' y devuelve una cadena vacía en lugar de esperar al usuario.

JAVA — El problema y la solución

```

// ❌ PROBLEMA: nextLine() lee el '\n' que dejó nextInt()
int edad = scanner.nextInt();
String nombre = scanner.nextLine(); // Devuelve cadena vacía!

// ✅ SOLUCIÓN: leer siempre con nextLine() y parsear
int edad = Integer.parseInt(scanner.nextLine());
String nombre = scanner.nextLine(); // Ahora funciona correctamente

```

✅ Solución recomendada

Leer siempre con **`nextLine()`** y convertir manualmente: **`Integer.parseInt(...)`**, **`Float.parseFloat(...)`**, etc. Es más seguro y consistente.

⚠ Errores comunes con Scanner

Usar `nextInt()` / `nextDouble()` seguido de `nextLine()` sin limpiar el buffer.

Olvidar importar **`java.util.Scanner`**.

No cerrar el Scanner al final: **`teclado.close();`**

4. Formateo de la salida con `printf()`

El método **`printf()`** permite imprimir datos con un formato preciso, controlando decimales, anchura de campos y alineación.

📖 Especificadores de formato

Códigos dentro de la cadena de printf() que indican cómo mostrar cada valor: **%s** para Strings, **%d** para enteros, **%f** para decimales, **%.2f** para 2 decimales, etc.

	Tipo	Descripción	Ejemplo de salida
%s	String	Cadena de texto	"Hola mundo"
%d	int	Entero decimal	"42"
%f	float/double	Decimal (6 decimales por defecto)	"3.141593"
%.2f	float/double	Decimal con 2 decimales	"3.14"
%5d	int	Entero en campo de 5 caracteres	" 42"
%-10s	String	Texto alineado a la izquierda	"Hola "
%04d	int	Entero relleno con ceros	"0007"
%n	—	Salto de línea (portable)	—

JAVA — Ejemplos de printf()

```
// Nombre y edad alineados
System.out.printf("Nombre: %-15s Edad: %3d años\n", "Blanca", 28);
// Salida: Nombre: Blanca      Edad:  28 años

// Precio con decimales
System.out.printf("Precio: %.2f€\n", 1234.5678);
// Salida: Precio: 1234.57€

// Tabla de multiplicar
for (int i = 1; i <= 5; i++)
    System.out.printf("%2d x 7 = %3d\n", i, i * 7);
```

✓ String.format()

Si necesitas guardar la cadena formateada en una variable usa **String.format()**:

JAVA

```
String msg = String.format("El total es: %.2f€", 29.9);
System.out.println(msg); // El total es: 29.90€
```

Bloque 5.2 · E/S basada en ficheros

1. Concepto de Flujo (Stream) en Java

Para leer o escribir ficheros en Java usamos el concepto de **flujo (stream)**: un canal de comunicación que transporta datos de manera secuencial entre el programa y el fichero.

Flujo (Stream)

Canal de comunicación que permite leer o escribir datos de manera secuencial. En Java, los flujos abstraen el origen o destino de los datos: fichero, memoria, red, etc.

Tipo de flujo	Clases principales	Uso recomendado
Basado en bytes	FileInputStream / FileOutputStream	Ficheros binarios (imágenes, audio...)
Basado en caracteres	FileReader / FileWriter	Ficheros de texto simples
Con buffer (rápido)	BufferedReader / BufferedWriter	Lectura/escritura de texto (recomendado)
Para parsing	Scanner + File	CSV, números, tokens

Analogía — El buffer como un camión de transporte

Leer un fichero sin buffer es como llevar ladrillos de uno en uno. El **BufferedReader** es como un camión: toma un lote grande de datos (8192 bytes) en una sola operación y los entrega al programa de forma mucho más eficiente.

2. Lectura de ficheros de texto

La manera más habitual de leer un fichero línea a línea es usar **BufferedReader** envolviendo a **FileReader**.

JAVA — Leer un fichero línea a línea

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

try (BufferedReader br = new BufferedReader(new FileReader("datos.txt"))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }
} catch (IOException e) {
    System.err.println("Error: " + e.getMessage());
}
```

try-with-resources

Construcción de Java (desde Java 7) que **cierra automáticamente los recursos** (ficheros, conexiones...) al final del bloque try, incluso si se produce una excepción.

JAVA — Leer números con Scanner

```
try (Scanner sc = new Scanner(new File("numeros.txt"))) {
    while (sc.hasNextInt()) {
        int n = sc.nextInt();
        System.out.println("Leído: " + n);
    }
} catch (FileNotFoundException e) {
    System.err.println("Fichero no encontrado: " + e.getMessage());
}
```

3. Escritura en ficheros de texto

Para escribir usamos **BufferedWriter** envolviendo a **FileWriter**. Con **true** como segundo parámetro de **FileWriter** el contenido se añade al final (modo append); sin él, se sobrescribe.

JAVA — Escribir y añadir al fichero

```
// Sobreescritura (sin true)
try (BufferedWriter bw = new BufferedWriter(new FileWriter("salida.txt"))) {
    bw.write("Primera línea");
    bw.newLine();
    bw.write("Segunda línea");
}

// Modo append: añadir al final (true)
try (BufferedWriter bw = new BufferedWriter(new FileWriter("log.txt", true))) {
    bw.write("Nueva entrada");
    bw.newLine();
}
```

4. Gestión de Excepciones en E/S

Las operaciones con ficheros pueden fallar por causas externas. Java obliga a gestionar estas situaciones con **try-catch**.

Excepción	Cuándo se produce
IOException	Error general de E/S (clase padre de las demás)
FileNotFoundException	El fichero especificado no existe
SecurityException	Permisos insuficientes para acceder al fichero

⚠ Siempre cerrar los recursos

Si no cerramos un fichero abierto se produce una **fuga de recursos**. Usa siempre **try-with-resources** para garantizar el cierre automático incluso si hay una excepción.

5. Ficheros CSV y creación de objetos

Un fichero **CSV (Comma-Separated Values)** guarda datos en texto plano donde cada línea es un registro y los campos están separados por comas. Es el formato que viste en el EAC5 con las apuestas.

📖 CSV — Comma-Separated Values

Formato de texto donde cada línea representa un registro y los campos están separados por comas. Ejemplo: **202405101530,Fútbol,Barça-Madrid,1.85,50.00**

JAVA — Leer un CSV y crear objetos

```
// Fichero productos.txt con líneas: nombre,precio
try (BufferedReader br = new BufferedReader(new FileReader("productos.txt"))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        String[] partes = linea.split(","); // Separar por comas
        if (partes.length == 2) {
            String nombre = partes[0].trim();
            float precio = Float.parseFloat(partes[1].trim());
            Producto p = new Producto(nombre, precio);
            System.out.println(p);
        }
    }
} catch (IOException | NumberFormatException e) {
    System.err.println("Error: " + e.getMessage());
}
```

✅ Buenas prácticas con ficheros CSV

Validar el número de campos (**partes.length**) antes de acceder a los índices.

Usar **.trim()** para eliminar espacios en blanco alrededor de los valores.

Capturar **NumberFormatException** si algún campo debería ser numérico.

Usar try-with-resources para garantizar el cierre del fichero.

6. Resumen — Comparativa de clases E/S

Clase	Propósito	Método clave	Cuándo usarla
BufferedReader	Leer texto eficientemente	readLine()	Ficheros de texto grandes
FileReader	Leer texto carácter a carácter	read()	Dentro de BufferedReader
BufferedWriter	Escribir texto eficientemente	write() + `newLine()`	Ficheros de texto
FileWriter	Escritura básica	write()	Dentro de BufferedWriter
Scanner + File	Leer y parsear datos	`nextLine()` etc.	CSV, números, tokens

⚠ Errores típicos a evitar

Olvidar el try-with-resources → fuga de recursos.

No gestionar IOException → el programa puede petar sin avisar.

Acceder a `partes[1]` sin comprobar `partes.length >= 2`.

FileWriter sin segundo parámetro cuando quieres añadir → sobrescribe el fichero.

Ejercicios de repaso · Unidad 5

Ejercicio 1 — Lectura desde el teclado (Fácil · 10 min)

Escribe un programa que pida nombre, apellido y edad y muestre la información en una línea: **Nombre completo: Blanca M. | Edad: 28 años**

- Crea una clase **DatosPersonales** con un método main.
- Instancia un Scanner: `Scanner sc = new Scanner(System.in);`
- Lee nombre, apellido y edad con `sc.nextLine() + Integer.parseInt()`.
- Imprime el resultado con `System.out.printf()`.
- Cierra el Scanner: `sc.close();`

Ejercicio 2 — Tabla de multiplicar con printf (Fácil · 15 min)

Escribe un programa que pida un número al usuario y muestre su tabla de multiplicar del 1 al 10, alineada con printf().

- Lee el número con `Integer.parseInt(sc.nextLine())`.
- Usa un bucle `for (int i = 1; i <= 10; i++)`.
- Imprime con: `System.out.printf("%2d x %2d = %3d%n", numero, i, numero*i);`

Ejercicio 3 — Escribir un fichero CSV (Medio · 20 min)

Escribe un programa que pida 3 productos (nombre y precio) y los guarde en **productos.csv**.

- Abre un BufferedWriter con `new BufferedWriter(new FileWriter("productos.csv"))` en un try-with-resources.
- Para cada producto lee nombre y precio, y escribe `bw.write(nombre + "," + precio)` seguido de `bw.newLine()`.
- Verifica que se ha creado el fichero con el contenido correcto.

Ejercicio 4 — Leer el fichero CSV (Medio · 20 min)

Lee el fichero **productos.csv** del ejercicio anterior, parsea cada línea y muestra los productos en formato de tabla.

- Abre con `new BufferedReader(new FileReader("productos.csv"))`.
- Lee línea a línea con `br.readLine()`.
- Usa `linea.split(",")` y comprueba que `partes.length == 2`.
- Imprime con: `System.out.printf("%-15s %6.2f€\n", nombre, precio);`

- Captura IOException y NumberFormatException.

UNIDAD 6

Estructuras de datos y Colecciones (RA6)

Arrays · Algoritmos de búsqueda y ordenación · Collections Framework

**Objetivos de aprendizaje de esta unidad**

Entender y manejar arrays unidimensionales y bidimensionales en Java.
Implementar algoritmos fundamentales de búsqueda y ordenación.
Conocer el Java Collections Framework: List, Set y Map.
Elegir la colección adecuada según el problema.
Usar Comparator para ordenaciones personalizadas.

Mapa de contenidos

Bloque 6.1 — Estructuras básicas	Bloque 6.2 — Colecciones avanzadas
• Arrays 1D y 2D • Búsqueda lineal y binaria • Bubble Sort, Selection Sort • Insertion Sort	• ArrayList vs LinkedList • HashSet / TreeSet • HashMap / TreeMap • Streams y Comparator

Bloque 6.1 · Estructuras de datos básicas — Arrays

1. ¿Qué es un array?

Estructura de datos más básica Java: una colección de elementos del mismo tipo, almacenados en posiciones de memoria contiguas y accesibles con un índice numérico.

Array — Definición

Estructura de datos que contiene una colección de elementos del mismo tipo. El índice empieza siempre en 0 y llega hasta longitud - 1.

Ventajas	Inconvenientes
Acc. directo $O(1)$ a cualquier elemento	Tamaño fijo: no se puede cambiar tras la creación
Almacenamiento contiguo, muy eficiente	Inserción/eliminación costosa: hay que desplazar elementos $O(n)$
Sencillos de entender e implementar	Todos los elementos deben ser del mismo tipo

2. Arrays unidimensionales

JAVA — Declaración, inicialización y acceso

```
int[] numeros = new int[5];           // Array de 5 enteros (todos a 0)
int[] valores = {1, 5, 10, -3, 8};    // Array inicializado

System.out.println(valores[0]);       // Muestra 1 (primer elemento)
valores[2] = 100;                    // Modifica la posición 2
System.out.println(valores.length);   // Longitud: 5
```

ArrayIndexOutOfBoundsException

Si intentas acceder a un índice fuera de rango (negativo o $\geq \text{length}$) el programa lanza esta excepción. Comprueba siempre que el índice esté entre 0 y $\text{array.length} - 1$.

2.1 Recorrido de un array

JAVA — For clásico y for-each

```
// For clásico (cuando necesitas el índice)
for (int i = 0; i < valores.length; i++) {
    System.out.print(valores[i] + " ");
}

// For-each (cuando solo necesitas el valor)
for (int v : valores) {
    System.out.print(v + " ");
}
```

2.2 Operaciones típicas

JAVA — Suma, máximo y búsqueda lineal

```
// Suma de todos los elementos
int suma = 0;
for (int i = 0; i < a.length; i++) suma += a[i];

// Máximo
int max = a[0];
for (int i = 1; i < a.length; i++)
    if (a[i] > max) max = a[i];

// Búsqueda lineal — devuelve el índice o -1 si no está
public static int buscar(int[] a, int n) {
    for (int i = 0; i < a.length; i++)
        if (a[i] == n) return i;
    return -1;
}
```

3. Arrays bidimensionales (matrices)

Un array bidimensional organiza los datos en filas y columnas. Se accede con dos índices: `[fila][columna]`.

Matriz — Definición

Array de arrays. `int[][] m = new int[3][4];` crea una matriz de 3 filas y 4 columnas.

JAVA — Declaración, acceso y recorrido

```
int[][] matriz = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

System.out.println(matriz[1][2]); // Muestra 6 (fila 1, columna 2)

for (int i = 0; i < matriz.length; i++) {
    for (int j = 0; j < matriz[i].length; j++)
        System.out.print(matriz[i][j] + " ");
    System.out.println();
}
```

4. Algoritmos de búsqueda

4.1 Búsqueda lineal

JAVA — Búsqueda lineal

```
public static int busquedaLineal(int[] array, int valor) {
    for (int i = 0; i < array.length; i++) {
        if (array[i] == valor) return i; // Encontrado
    }
    return -1; // No encontrado
}
```

}

📖 Búsqueda Lineal — Complejidad $O(n)$

En el peor caso hay que recorrer todos los elementos. Con 1000 elementos, puede hacer hasta 1000 comparaciones.

4.2 Búsqueda binaria

Solo funciona con arrays ordenados. En cada paso descarta la mitad de los elementos restantes.

JAVA — Búsqueda binaria

```
public static int busquedaBinaria(int[] array, int valor) {
    int izq = 0, der = array.length - 1;
    while (izq <= der) {
        int mid = (izq + der) / 2;
        if (array[mid] == valor) return mid;           // Encontrado
        else if (array[mid] < valor) izq = mid + 1;    // Buscar derecha
        else der = mid - 1;                           // Buscar izquierda
    }
    return -1;
}
```

📖 Búsqueda Binaria — Complejidad $O(\log n)$

Con 1000 elementos, necesita como máximo ~10 comparaciones. Requiere que el array esté ordenado.

5. Algoritmos de ordenación**5.1 Bubble Sort (burbuja)**

JAVA — Bubble Sort

```
public static void bubbleSort(int[] a) {
    for (int i = 0; i < a.length - 1; i++) {
        boolean cambiado = false;
        for (int j = 0; j < a.length - 1 - i; j++) {
            if (a[j] > a[j+1]) {
                int tmp = a[j]; a[j] = a[j+1]; a[j+1] = tmp;
                cambiado = true;
            }
        }
        if (!cambiado) break; // Ya ordenado, salir antes
    }
}
```

5.2 Selection Sort (selección)

JAVA — Selection Sort

```
public static void selectionSort(int[] a) {
    for (int i = 0; i < a.length - 1; i++) {
        int idxMin = i;
```

```

        for (int j = i+1; j < a.length; j++)
            if (a[j] < a[idxMin]) idxMin = j;
        if (idxMin != i) {
            int tmp = a[i]; a[i] = a[idxMin]; a[idxMin] = tmp;
        }
    }
}

```

5.3 Insertion Sort (inserción)

JAVA — Insertion Sort

```

public static void insertionSort(int[] a) {
    for (int i = 1; i < a.length; i++) {
        int clave = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > clave) {
            a[j+1] = a[j];
            j--;
        }
        a[j+1] = clave;
    }
}

```

Algoritmo	Mejor caso	Caso medio	Peor caso	¿Cuándo usarlo?
Búsqueda Lineal	$O(1)$	$O(n)$	$O(n)$	Array sin ordenar
Búsqueda Binaria	$O(1)$	$O(\log n)$	$O(\log n)$	Array ordenado
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Didáctico / casi ordenado
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Didáctico
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Arrays pequeños o casi ordenados

✓ En la práctica — usa `Arrays.sort()`

Java incluye un método ya optimizado: `Arrays.sort(miArray);`. Los algoritmos anteriores se estudian para entender la lógica, pero en proyectos reales usa siempre `Arrays.sort()`.

Bloque 6.2 · Colecciones avanzadas — Java Collections Framework

1. ¿Qué es el Java Collections Framework?

El **Java Collections Framework** (JCF) proporciona estructuras de datos dinámicas y algoritmos para manipularlas de forma eficiente.

Característica	Arrays	Colecciones (JCF)
Tamaño	Fijo (definido al crear)	Dinámico (crece y decrece)
Tipos de datos	Primitivos y objetos	Solo objetos (con autoboxing)
Operaciones	Básicas (acceso por índice)	Avanzadas (búsqueda, orden...)
Duplicados	Permitidos siempre	Depende de la colección

💡 Las tres interfaces principales del JCF

List — Colección ordenada, permite duplicados. Acceso por índice.

Set — Colección sin duplicados. No garantiza orden (salvo TreeSet).

Map — Pares clave→valor. Cada clave es única. No hereda de Collection.

2. Interfaz List — ArrayList y LinkedList

2.1 ArrayList

JAVA — ArrayList básico

```
ArrayList<String> nombres = new ArrayList<>();

nombres.add("Ana");           // Añadir al final
nombres.add(1, "Bea");        // Insertar en posición 1
nombres.set(0, "Alba");        // Modificar posición 0
nombres.remove(1);             // Eliminar por índice
nombres.remove("Bea");         // Eliminar por valor

System.out.println(nombres.get(0)); // Acceso por índice
System.out.println(nombres.size());  // Tamaño
System.out.println(nombres.contains("Ana")); // ¿Contiene?

for (String n : nombres) System.out.println(n);
```

2.2 LinkedList

JAVA — LinkedList

```
LinkedList<String> tareas = new LinkedList<>();
tareas.add("Tarea 1");
tareas.addFirst("Urgente"); // Añadir al principio
tareas.addLast("Opcional"); // Añadir al final

System.out.println(tareas.getFirst()); // Primer elemento
System.out.println(tareas.getLast());  // Último elemento
tareas.removeFirst();                  // Eliminar el primero
```

Operación	ArrayList	LinkedList	¿Cuál elegir?
Acceso por índice	O(1)	O(n)	ArrayList
Añadir al final	O(1)*	O(1)	Empate
Insertar al principio	O(n)	O(1)	LinkedList
Memoria por elemento	Solo objeto	Objeto + 2 refs	ArrayList

Regla práctica

Usa **ArrayList** por defecto. Cambia a **LinkedList** solo si hay muchas inserciones/eliminaciones al principio o actúas como cola/pila.

3. Interfaz Set — HashSet y TreeSet

3.1 HashSet — sin orden, O(1)

JAVA — HashSet

```
HashSet<String> ciudades = new HashSet<>();
ciudades.add("Barcelona");
ciudades.add("Madrid");
ciudades.add("Barcelona"); // Duplicado ignorado automáticamente

System.out.println(ciudades.size()); // 2

// Eliminar duplicados de una lista
List<Integer> lista = Arrays.asList(1,2,2,3,3,4);
Set<Integer> sinDups = new HashSet<>(lista);
System.out.println(sinDups); // [1, 2, 3, 4]
```

3.2 TreeSet — ordenado, O(log n)

JAVA — TreeSet

```
TreeSet<Integer> nums = new TreeSet<>();
nums.add(50); nums.add(20); nums.add(80); nums.add(10);

System.out.println(nums); // [10, 20, 50, 80] - siempre ordenado
System.out.println(nums.first()); // 10
System.out.println(nums.headSet(40)); // [10, 20]
```

⚠ Para usar objetos propios en HashSet

Debes implementar `equals()` y `hashCode()` en tu clase. Si no, el Set no detectará duplicados correctamente.

4. Interfaz Map — HashMap y TreeMap

4.1 HashMap — O(1), sin orden

JAVA — HashMap

```
Map<String, String> trad = new HashMap<>();
trad.put("hello", "hola");
trad.put("world", "mundo");

String t = trad.get("hello"); // "hola"
trad.containsKey("world"); // true

for (Map.Entry<String, String> e : trad.entrySet())
    System.out.println(e.getKey() + " -> " + e.getValue());

// Contar frecuencia de palabras
Map<String, Integer> freq = new HashMap<>();
for (String p : palabras)
    freq.put(p, freq.getOrDefault(p, 0) + 1);
```

4.2 TreeMap — claves ordenadas, $O(\log n)$

JAVA — TreeMap

```
TreeMap<String, Integer> cont = new TreeMap<>();
cont.put("gato", 3); cont.put("perro", 1); cont.put("aves", 2);

// Siempre en orden alfabético de claves
cont.forEach((k, v) -> System.out.println(k + ": " + v));
// aves: 2   gato: 3   perro: 1
```

Implementación	Orden	get/put	Cuándo usar
HashMap	Sin orden	$O(1)$ medio	Rendimiento general (la más usada)
TreeMap	Claves ordenadas	$O(\log n)$	Cuando necesitas las claves en orden
LinkedHashMap	Orden de inserción	$O(1)$ medio	Cuando necesitas recordar el orden de inserción

5. Algoritmos con colecciones y Comparator

JAVA — Clase Collections

```
Collections.sort(nums); // [1, 2, 5, 8, 9]
Collections.shuffle(nums); // Mezclar aleatoriamente
Collections.reverse(nums); // Invertir orden
int max = Collections.max(nums); // Máximo
int pos = Collections.binarySearch(nums, 5); // Búsqueda binaria (array ordenado)
```

5.1 Comparator

JAVA — Ordenación personalizada

```
// Por edad ascendente
personas.sort(Comparator.comparing(p -> p.getEdad()));

// Por edad descendente
personas.sort(Comparator.comparing(Persona::getEdad).reversed());

// Múltiple: primero edad, luego nombre
personas.sort(Comparator.comparing(Persona::getEdad)
    .thenComparing(Persona::getNombre));
```

5.2 Streams básicos

JAVA — Streams

```
// Filtrar los que empiezan por 'A'
List<String> filtrados = nombres.stream()
    .filter(n -> n.startsWith("A"))
    .collect(Collectors.toList());

// Suma de cuadrados de números pares
int suma = nums.stream()
    .filter(n -> n % 2 == 0)
    .mapToInt(n -> n * n)
    .sum(); // 4 + 16 + 36 = 56
```

⚠ Errores comunes con colecciones

Modificar una colección dentro de un for-each →

`ConcurrentModificationException`.

Usar `TreeSet` o `TreeMap` sin necesitar orden → más lento sin motivo.

Objetos propios en `HashSet/HashMap` sin implementar `equals()` y `hashCode()`.

Confundir `remove(int índice)` con `remove(Object valor)` en listas de Integer.

Ejercicios de repaso · Unidad 6

Ejercicio 1 — Invertir un array (Fácil · 15 min)

Método que invierta un array de enteros sin usar colección auxiliar. `[1,2,3,4,5] → [5,4,3,2,1]`.

- Usa dos punteros: `izq = 0` y `der = array.length - 1`.
- Mientras `izq < der`: intercambia `array[izq]` con `array[der]`.
- Incrementa `izq` y decrementa `der`.
- Imprime el resultado con `Arrays.toString(array)`.

Ejercicio 2 — Bubble Sort paso a paso (Fácil · 20 min)

Implementa **Bubble Sort** y muestra el estado del array después de cada pasada.

- Implementa el método `bubbleSort(int[] a)`.
- Después de cada iteración del bucle externo, imprime el array.
- Prueba con `{64, 34, 25, 12, 22, 11, 90}`.
- Comprueba que el flag `cambiado` reduce el número de pasadas.

Ejercicio 3 — Lista de la compra con **ArrayList** (Fácil · 20 min)

Programa que gestione lista de la compra con **ArrayList**: añadir, eliminar, buscar y mostrar ordenada.

- Crea un `ArrayList<String> listaCompra`.
- Añade al menos 5 productos.
- Elimina uno por nombre con `remove("producto")`.
- Comprueba si un producto está con `contains()`.
- Muestra la lista ordenada con `Collections.sort(listaCompra)`.

Ejercicio 4 — Contador de palabras con **HashMap** (Medio · 25 min)

Lee un texto **hardcodeado** y cuenta cuántas veces aparece cada palabra usando `HashMap`.

- Define un String con palabras repetidas.
- Separa: `String[] palabras = texto.split(" ");`
- Para cada palabra: `freq.put(p, freq.getOrDefault(p, 0) + 1);`

- Muestra las entradas con `freq.entrySet()`.
- Bonus: ordena por frecuencia con `Comparator`.

Ejercicio 5 — Eliminar duplicados con `LinkedHashSet` (Medio · 15 min)

Dado un array con números repetidos, elimina los duplicados manteniendo el orden original.

- Parte de: `int[] nums = {3, 1, 4, 1, 5, 9, 2, 6, 5, 3};`
- Convierte a lista con `Arrays.asList(...)`.
- Crea `LinkedHashSet<Integer> set = new LinkedHashSet<>(lista);`
- Convierte de vuelta a lista y muestra el resultado.
- ¿Por qué `LinkedHashSet` y no `HashSet`?

UNIDAD 7

Utilización avanzada de clases (RA7)

POO · Herencia · Polimorfismo · Clases abstractas · Interfaces · SOLID · Patrones

💡 Objetivos de aprendizaje de esta unidad

Entender los 4 pilares de la POO: encapsulación, herencia, polimorfismo y abstracción.

Usar herencia con **extends**, **super** y **@Override**.

Aplicar polimorfismo e instanceof para tratar objetos de forma uniforme.

Distinguir y usar clases abstractas e interfaces.

Conocer los 5 principios SOLID y los patrones Singleton, Observer y Strategy.

Mapa de contenidos

Bloque 7.1 — Herencia y polimorfismo	Bloque 7.2 — Abstracción avanzada
• Clases y objetos • Modificadores de acceso • Herencia (extends / super) • @Override y polimorfismo • instanceof y casting • Composición vs herencia	• Clases abstractas • Interfaces • Abstracta vs interfaz • Principios SOLID (5) • Patrones: Singleton, Observer, Strategy

Bloque 7.1 · Herencia, polimorfismo y composición

1. Los 4 pilares de la POO

La **POO** organiza el código en clases y objetos para mejorar la abstracción, reutilización y mantenimiento.

Los 4 pilares fundamentales de la POO

Encapsulación — Agrupar datos y métodos relacionados, protegiendo el estado interno con modificadores de acceso.

Herencia — Crear nuevas clases basadas en clases existentes, reutilizando y extendiendo su comportamiento.

Polimorfismo — Un mismo método se comporta de manera diferente según el tipo real del objeto.

Abstracción — Ocultar la complejidad interna y mostrar solo lo necesario al exterior.

2. Clases y objetos

Una clase es una plantilla que define atributos y métodos. Un objeto es una instancia concreta de esa clase con valores reales.

Clase y Objeto

Clase: plantilla que define qué atributos y métodos tendrán sus objetos. Como un plano de construcción.

Objeto: instancia concreta de una clase con valores reales en sus atributos.

JAVA — Clase Persona con constructor, getters y setters

```
public class Persona {
    private String nombre;
    private int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    public String getNombre() { return nombre; }

    public void setEdad(int edad) {
        if (edad >= 0) this.edad = edad;
    }

    public void mostrarInfo() {
        System.out.println("Nombre: " + nombre + " | Edad: " + edad);
    }
}
```

Modificador	Clase	Paquete	Subclase	Todos	Uso típico
private	✓	✗	✗	✗	Atributos internos
(default)	✓	✓	✗	✗	Clases del mismo paquete
protected	✓	✓	✓	✗	Herencia
public	✓	✓	✓	✓	API pública

3. La palabra clave this

La palabra **clave** `this` hace referencia al objeto actual. Se usa principalmente para distinguir atributos de parámetros con el mismo nombre y para encadenar constructores.

JAVA — Usos de `this`

```
public Cotxe(String marca, int any) {
    this.marca = marca; // this distingue atributo de parámetro
    this.any = any;
}

// Constructor encadenado: llama a otro constructor de la misma clase
public Cotxe(String marca) {
    this(marca, 2024); // llama al constructor de arriba
}

// Retornar this permite encadenar métodos (fluent interface)
public Cotxe setAny(int any) {
    this.any = any;
    return this; // cotxe.setAny(2023).mostrarInfo()
}
```

4. Herencia

La herencia permite crear nuevas clases basadas en clases existentes. La subclase hereda atributos y métodos del padre y puede añadir o modificar comportamiento.

Herencia — Conceptos clave

Superclase (clase padre): la clase de la que se hereda.

Subclase (clase hija): la clase que hereda, usa la palabra clave `extends`.

super: referencia a la superclase — para llamar a su constructor o sus métodos.

@Override: anotación que indica que se sobrescribe un método del padre.

JAVA — Superclase `Animal` y subclase `Perro`

```
public class Animal {
    protected String nombre;

    public Animal(String nombre) { this.nombre = nombre; }
    public void comer() { System.out.println(nombre + " come"); }
    public void mostrarInfo() { System.out.println("Animal: " + nombre); }
}

public class Perro extends Animal {
```

```

private String raza;

public Perro(String nombre, String raza) {
    super(nombre); // llama al constructor de Animal
    this.raza = raza;
}

public void ladrar() { System.out.println(nombre + " ¡Guau!"); }

@Override
public void mostrarInfo() {
    super.mostrarInfo(); // llama al método del padre
    System.out.println("Raza: " + raza);
}
}

```

⚠ Reglas de @Override

El método debe tener la misma firma (nombre, parámetros y tipo de retorno).

No puede ser más restrictivo en accesibilidad (no puedes hacer private un método public).

Los métodos **final**, **static** y **private** no se pueden sobrescribir.

5. Polimorfismo

El **polimorfismo** permite que una variable de tipo superclase apunte a objetos de subclases, y que cada objeto responda a los métodos con su propia implementación sobrescrita.

📖 Polimorfismo — Definición

Capacidad de los objetos de responder de manera diferente al mismo método según su tipo real. Permite tratar objetos de distintas clases de forma uniforme a través de una referencia de la superclase.

JAVA — Polimorfismo e instanceof

```

// Variable de tipo Animal apunta a objetos Perro/Gato
Animal a1 = new Perro("Rex", "Labrador");
Animal a2 = new Gato("Luna", true);

// Array polimórfico – cada animal ejecuta su versión del método
Animal[] animales = {a1, a2};
for (Animal a : animales) {
    a.mostrarInfo(); // llama a la versión de Perro o Gato
}

// instanceof + casting para acceder a métodos específicos
if (a1 instanceof Perro) {
    Perro p = (Perro) a1; // casting seguro
    p.ladrar(); // método específico de Perro
}

```

6. Composición vs Herencia

La **composición** construye clases usando objetos de otras clases como atributos internos (relación 'tiene-un'). Es una alternativa más flexible a la herencia.

Aspecto	Herencia (es-un)	Composición (tiene-un)
Relación	Perro es un Animal	Coche tiene un Motor
Acoplamiento	Fuerte (depende del padre)	Débil (clases independientes)
Flexibilidad	Limitada	Alta (cambiar componentes en runtime)
Reutilización	Por herencia	Por delegación

JAVA — Composición: Coche tiene un Motor

```
class Motor {
    private String tipo;
    public Motor(String tipo) { this.tipo = tipo; }
    public void arrancar() { System.out.println("Motor " + tipo + "
arrancando"); }
}

class Coche {
    private String marca;
    private Motor motor; // composición: Coche TIENE UN Motor

    public Coche(String marca, Motor motor) {
        this.marca = marca; this.motor = motor;
    }

    public void arrancar() {
        System.out.println(marca + " arrancando:");
        motor.arrancar(); // delega al componente
    }

    public void cambiarMotor(Motor nuevo) {
        this.motor = nuevo; // flexibilidad en runtime ✅
    }
}
```

💡 ¿Herencia o composición?

Herencia cuando existe una relación clara 'es-un' (Perro es un Animal) y quieres aprovechar el polimorfismo.

Composición cuando existe una relación 'tiene-un' o necesitas cambiar comportamiento en tiempo de ejecución.

Principio general: "Favorece la composición sobre la herencia" para sistemas más flexibles.

1. Clases abstractas

Una **clase abstracta** no puede instanciarse directamente. Define una plantilla para sus subclases, mezclando métodos concretos (con implementación) y abstractos (sin implementación).

Clase abstracta — Características

Usa la palabra clave **abstract** en la declaración de la clase.

No se puede instanciar con **new** directamente.

Puede tener métodos concretos (con cuerpo) Y métodos abstractos (sin cuerpo).

Las subclases deben implementar todos los métodos abstractos.

Puede tener constructores, atributos y métodos estáticos.

JAVA — Clase abstracta Figura y subclase Rectangulo

```
public abstract class Figura {
    protected String color;

    public Figura(String color) { this.color = color; }

    public String getColor() { return color; } // método concreto

    public abstract double calcularArea();      // sin cuerpo – subclases
    deben implementarlo
    public abstract double calcularPerimetro();
}

public class Rectangulo extends Figura {
    private double ancho, alto;

    public Rectangulo(String color, double ancho, double alto) {
        super(color); this.ancho = ancho; this.alto = alto;
    }

    @Override public double calcularArea() { return ancho * alto; }
    @Override public double calcularPerimetro() { return 2 * (ancho + alto); }
}
```

JAVA — Uso polimórfico

```
// ❌ No se puede instanciar una clase abstracta
// Figura f = new Figura("Rojo"); → Error de compilación

// ✅ Sí se puede usar como tipo de referencia
Figura[] figuras = {new Rectangulo("Rojo", 5, 3), new Circulo("Azul", 4)};
for (Figura f : figuras) // polimorfismo
    System.out.printf("Área: %.2f\n", f.calcularArea());
```

2. Interfaces

Una interfaz define un contrato: especifica qué métodos debe implementar una clase, sin proporcionar la implementación (salvo métodos **default** desde Java 8). Una clase puede implementar múltiples interfaces.

Interfaz — Características clave

Todos los métodos son implícitamente **public abstract** (Java 7-).

Desde Java 8: permite métodos **default** (con cuerpo) y **static**.

Los atributos son implícitamente **public static final** (constantes).

Una clase puede implementar múltiples interfaces (herencia múltiple de comportamiento).

JAVA — Interfaces múltiples

```
public interface Dibujable {
    void dibujar();
    void mover(int x, int y);
    default void mostrarInfo() { System.out.println("Objeto dibujable"); } //
    Java 8+
}

public interface Redimensionable {
    void redimensionar(double factor);
    double getArea();
}

// Una clase implementa MÚLTIPLES interfaces a la vez
public class Rectangulo implements Dibujable, Redimensionable {
    private double ancho, alto, x, y;

    @Override
    public void dibujar() { System.out.println("Rect " + ancho + "x" + alto); }

    @Override
    public void mover(int x, int y) { this.x=x; this.y=y; }

    @Override
    public void redimensionar(double f) { ancho *= f; alto *= f; }

    @Override
    public double getArea() { return ancho * alto; }
}
```

3. Clase abstracta vs Interfaz

Aspecto	Clase abstracta	Interfaz
Herencia	Solo una (<code><code>extends</code></code>)	Múltiples (<code><code>implements</code></code>)
Métodos	Concretos + abstractos	Abstractos + default + static
Atributos	Cualquier tipo	Solo constantes (public static final)
Constructores	Sí	No
Cuándo usar	Relación 'es-un' con código común	Contrato de comportamiento

💡 Regla práctica

Clase abstracta cuando quieras compartir código entre clases relacionadas y necesites atributos no constantes.

Interfaz cuando quieras definir un contrato de comportamiento que varias clases no relacionadas puedan implementar.

4. Principios SOLID

Los **principios SOLID** son 5 guías de diseño orientado a objetos para crear código más mantenible, flexible y robusto.

Letra	Principio	En una frase
S	Single Responsibility (SRP)	Una clase, una sola razón para cambiar
O	Open/Closed (OCP)	Abierto para extender, cerrado para modificar
L	Liskov Substitution (LSP)	Una subclase debe poder sustituir a su padre sin romper nada
I	Interface Segregation (ISP)	No obligues a implementar lo que no se usa
D	Dependency Inversion (DIP)	Depende de abstracciones, no de implementaciones concretas

S — Single Responsibility Principle (SRP)

JAVA — SRP: separar responsabilidades

```
// ❌ Violación: una clase hace demasiadas cosas
class GestorUsuario { validar(); guardarEnBD(); enviarEmail(); }

// ✅ Correcto: una responsabilidad por clase
class ValidadorUsuario { boolean esValido(Usuario u); }
class RepositorioUsuario { void guardar(Usuario u); }
class ServicioEmail { void enviarBienvenida(Usuario u); }
```

O — Open/Closed Principle (OCP)

JAVA — OCP: extensible sin modificar

```
//  Hay que modificar la clase para añadir nuevas formas
class Calculadora {
    double area(Object f) {
        if (f instanceof Rectangulo) { ... } // para añadir Triangulo hay que
        editar aquí 
    }
}

//  Interfaz permite añadir tipos SIN modificar el calculador
interface Forma { double calcularArea(); }
class Rectangulo implements Forma { /* ... */ }
class Triangulo implements Forma { /* añadida SIN modificar nada  */ }

class CalculadoraOCP {
    double area(Forma f) { return f.calcularArea(); } // nunca cambia
}
```

D — Dependency Inversion Principle (DIP)

JAVA — DIP: depender de abstracciones

```
//  Violación: Notificador acoplado a Gmail concreto
class Notificador { private Gmail g = new Gmail(); }

//  Correcto: depende de una interfaz, no de una clase concreta
interface Notificacion { void enviar(String dest, String msg); }
class Email implements Notificacion { /* ... */ }
class SMS implements Notificacion { /* ... */ }

class Notificador { // depende de la abstracción, no de Email ni SMS
    private Notificacion svc;
    public Notificador(Notificacion s) { this.svc = s; }
    public void notificar(String u, String m) { svc.enviar(u, m); }
}
```

5. Patrones de diseño

Los **patrones de diseño** son soluciones reutilizables a problemas comunes de diseño. Hay tres familias: creacionales (cómo crear objetos), estructurales (cómo componerlos) y de comportamiento (cómo interactúan).

5.1 Singleton (Creacional)

Garantiza que una clase tenga una sola instancia y proporciona un punto de acceso global.

JAVA — Singleton thread-safe

```
public class Configuracion {
    private static volatile Configuracion instancia;
    private String bdUrl = "localhost:5432/app";

    private Configuracion() {} // privado: nadie puede hacer new
    Configuracion()

    public static Configuracion getInstance() {
        if (instancia == null) {
```

```

        synchronized (Configuracion.class) {
            if (instancia == null) // doble chequeo para thread safety
                instancia = new Configuracion();
        }
    }
    return instancia;
}
public String getBdUrl() { return bdUrl; }
}

// Siempre la misma instancia
Configuracion c1 = Configuracion.getInstance();
Configuracion c2 = Configuracion.getInstance();
System.out.println(c1 == c2); // true

```

5.2 Observer (Comportamiento)

Cuando el sujeto observable cambia de estado, notifica automáticamente a todos sus observadores.

JAVA — Observer

```

interface Observador { void actualizar(double temp); }

class Sensor { // sujeto observable
    private List<Observador> obs = new ArrayList<>();
    private double temp;

    public void addObserver(Observador o) { obs.add(o); }
    public void setTemp(double t) {
        this.temp = t;
        obs.forEach(o -> o.actualizar(t)); // notifica a todos
    }
}

class Display implements Observador {
    @Override
    public void actualizar(double t) { System.out.println("Temp: " + t); }
}

```

5.3 Strategy (Comportamiento)

Define una familia de algoritmos intercambiables. Permite cambiar el algoritmo en tiempo de ejecución sin modificar el código que lo usa.

JAVA — Strategy

```

interface EstrategiaSort { void ordenar(int[] a); }

class BubbleSort implements EstrategiaSort {
    @Override
    public void ordenar(int[] a) { /* implementación bubble sort */ }
}

class QuickSort implements EstrategiaSort {
    @Override
    public void ordenar(int[] a) { /* implementación quicksort */ }
}

class Ordenador { // contexto que usa la estrategia
    private EstrategiaSort estrategia;
    public Ordenador(EstrategiaSort e) { this.estrategia = e; }
    public void setEstrategia(EstrategiaSort e) { this.estrategia = e; }
}

```

```

    public void ordenar(int[] a) { estrategia.ordenar(a); }
}

Ordenador ord = new Ordenador(new BubbleSort());
ord.ordenar(array);
ord.setEstrategia(new QuickSort()); // cambio transparente ✅
ord.ordenar(array);

```

✅ Ventajas de los patrones de diseño

Soluciones probadas a problemas comunes — no reinventas la rueda.
 Mejoran la comunicación entre desarrolladores ('usemos un Singleton aquí' es claro).
 Promueven los principios SOLID: bajo acoplamiento, alta cohesión.

Ejercicios de repaso · Unidad 7

Ejercicio 1 — Herencia básica (Fácil · 15 min)

Crea una jerarquía Animal → Perro/Gato usando herencia, `super()` y `@Override`.

Crea la clase Animal con atributo `nombre` y método `hacerSonido()`.

- Crea Perro que extienda Animal, añade atributo `raza` y sobrescribe `hacerSonido()`.
- Crea Gato igual que Perro pero con `esInterior`.
- En el constructor de Perro/Gato llama a `super(nombre)`.
- Crea un array `Animal[]` con objetos Perro y Gato y recórrelo polimórficamente.

Ejercicio 2 — Polimorfismo e instanceof (Fácil · 15 min)

Con la jerarquía anterior, practica el polimorfismo y el casting seguro.

- Crea un array `Animal[]` mezclando Perros y Gatos.
- Recórrelo llamando a `hacerSonido()` en cada elemento.
- Dentro del bucle, usa `instanceof` para detectar el tipo real.
- Si es Perro, haz casting y llama a un método específico de Perro.
- Imprime cuántos Perros y cuántos Gatos hay en el array.

Ejercicio 3 — Clase abstracta e interfaz (Medio · 20 min)

Implementa la jerarquía de figuras geométricas con clase abstracta e interfaz.

- Crea la clase abstracta Figura con métodos abstractos `calcularArea()` y `calcularPerimetro()`.
- Crea la interfaz Dibujable con método `dibujar()`.
- Implementa Rectangulo y Circulo que extiendan Figura e implementen Dibujable.
- Crea un array de Figuras y calcula el área total de forma polimórfica.
- ¿Puedes instanciar Figura directamente? Explica por qué.

Ejercicio 4 — Principio SRP (Fácil · 10 min)

Separa las responsabilidades de esta clase: `class Usuario { validar(); guardarBD(); enviarEmail(); }`

- Crea la clase Usuario solo con atributos nombre y email.
- Crea ValidadorUsuario con `esValido(Usuario u)`.
- Crea RepositorioUsuario con `guardar(Usuario u)`.
- Crea ServicioEmail con `enviarBienvenida(Usuario u)`.
- Crea ServicioUsuario que coordine los tres anteriores.

Ejercicio 5 — Patrón Observer (Medio · 20 min)

Implementa un termostato observable que notifique cambios de temperatura.

- Crea la interfaz Observador con `actualizar(double temp)`.
- Crea Termostato con lista de observadores y método `setTemperatura()`.
- Implementa DisplayTemperatura que imprima la temperatura recibida.
- Implementa AlertaTemperatura que imprima alerta si la temperatura supera 35°C.
- Prueba añadiendo/eliminando observadores y cambiando la temperatura.

UNIDAD 8

Mantenimiento y gestión de bases de datos (RA8/RA9)

ORM · Hibernate · JPA · Patrón DAO · CRUD con PostgreSQL

**Objetivos de aprendizaje de esta unidad**

Entender qué es un ORM y por qué usar Hibernate en lugar de JDBC manual.
Conocer los componentes clave de Hibernate: SessionFactory, Session, Transaction.
Mapear entidades Java a tablas con anotaciones JPA (**@Entity**, **@Id**, **@Column**...).
Implementar el patrón DAO para encapsular el acceso a datos.
Realizar operaciones CRUD completas con Hibernate y PostgreSQL.

Mapa de contenidos

Bloque 8.1 — Fundamentos ORM	Bloque 8.2 — Proyecto completo
• ORM vs JDBC manual • Componentes de Hibernate • Anotaciones JPA • Sesiones y transacciones • Patrón DAO	• Instalación del entorno • Proyecto Maven • hibernate.cfg.xml • HibernateUtil, Producte, DAO • CRUD + verificación PostgreSQL

Bloque 8.1 · Fundamentos de persistencia y ORM

1. ¿Qué es la persistencia y un ORM?

Las aplicaciones necesitan guardar datos más allá de la ejecución del programa. La persistencia consiste en almacenar el estado de los objetos Java en una base de datos relacional de forma segura y consistente.

ORM — Object-Relational Mapping

Técnica que automatiza la correspondencia entre el modelo de objetos de una aplicación y el esquema relacional de una base de datos, eliminando gran parte del SQL explícito.

¿Por qué Hibernate?

Hibernate es el ORM más usado en el ecosistema Java. Implementa la especificación JPA (Jakarta Persistence API), soporta múltiples bases de datos (PostgreSQL, MySQL, Oracle...) y ofrece gestión avanzada de caché, validación y transacciones.

2. JDBC manual vs Hibernate

Sin ORM, el desarrollador trabaja con filas y columnas. Cada consulta requiere construir SQL, gestionar recursos y transformar resultados en objetos manualmente.

JAVA — Sin Hibernate: código orientado a tablas (JDBC)

```
String sql = "SELECT id, nom, preu FROM productes WHERE id = ?";
try (Connection conn = DriverManager.getConnection(url, user, pass);
    PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setLong(1, 42L);
    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) {
            Long id = rs.getLong("id");
            String nom = rs.getString("nom");
            // hay que crear y rellenar el objeto Producte manualmente
        }
    }
}
```

Con Hibernate el mismo resultado se obtiene trabajando con objetos, sin SQL explícito:

JAVA — Con Hibernate: código orientado a objetos

```
public void actualizarPrecio(Long id, BigDecimal nuevoPrecio) {
    Producte prod = producteDAO
        .findById(id)
        .orElseThrow(() -> new IllegalArgumentException("Producto no
    encontrado"));

    prod.setPreu(nuevoPrecio);
    producteDAO.save(prod); // Hibernate genera el SQL automáticamente
}
```

Aspecto	JDBC manual	Hibernate / JPA
Cantidad de código	Mucho — SQL + mapeado manual	Poco — solo lógica de negocio
Cambios de esquema	Hay que actualizar todo el SQL	Solo se modifica la entidad
Conexiones/ transacciones	El dev las gestiona manualmente	Hibernate las encapsula
Portabilidad BD	SQL específico de cada BD	Cambia dialecto, no código

3. Componentes clave de Hibernate

Componente	Función	Cuándo intervenir
SessionFactory	Fábrica inmutable que gestiona configuración y crea sesiones	Al arrancar la aplicación — crear UNA sola vez
Session	Contexto de interacción: carga, guarda y sincroniza entidades	En cada operación de negocio
Transaction	Garantiza atomicidad agrupando operaciones (ACID)	Cuando se requiere all-or-nothing
Configuration	Carga propiedades y mapeos desde hibernate.cfg.xml	Al preparar el entorno

⚠ Una sola SessionFactory

Crear múltiples SessionFactory es muy costoso y puede saturar la base de datos. Constrúyela una única vez al inicio y reutilízala en toda la aplicación (patrón Singleton).

4. Mapeo de entidades con anotaciones JPA

La clave de cualquier ORM es describir cómo los objetos Java se corresponden con las tablas. Hibernate usa anotaciones JPA para definir entidades, claves primarias y relaciones.

📖 Entidad JPA

Clase Java persistente que se corresponde con una tabla de la base de datos. Cada instancia de la clase representa una fila de la tabla.

JAVA — Entidad Producte con anotaciones JPA

```
import jakarta.persistence.*;
import java.math.BigDecimal;

@Entity
@Table(name = "productes")
public class Producte {

    @Id
```

```

@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@Column(name = "nom", nullable = false, length = 100)
private String nom;

@Column(name = "preu", precision = 10, scale = 2)
private BigDecimal preu;

@Column(name = "stock")
private Integer stock;

// constructores, getters y setters...
}

```

Anotación	Para qué sirve
@Entity	Marca la clase como entidad JPA persistente
@Table	Especifica el nombre de la tabla (y esquema si hace falta)
@Id	Marca el atributo como clave primaria
@GeneratedValue	Auto-genera el valor de la PK (IDENTITY = autoincrement)
@Column	Configura nombre, longitud, nulidad y precisión del campo
@ManyToOne	Relación muchos-a-uno con clave foránea
@OneToMany	Relación uno-a-muchos (lado inverso de la FK)

5. Sesiones y transacciones

Una sesión actúa como unidad de trabajo. Agrupa operaciones que comparten el mismo contexto. Cuando se cierra, Hibernate sincroniza los cambios detectados (**dirty checking**) con la base de datos.

JAVA — Gestión de sesión y transacción

```

Transaction tx = null;
try (Session session = sessionFactory.openSession()) {
    tx = session.beginTransaction();

    Producte prod = session.get(Producte.class, 42L);
    prod.setPreu(new BigDecimal("24.90"));
    // Hibernate detecta el cambio automáticamente

    session.persist(prod);
    tx.commit(); // guarda los cambios de forma atómica
} catch (Exception ex) {
    if (tx != null) tx.rollback(); // deshace todo si falla
    throw ex;
}

```

Dirty checking

Mecanismo de Hibernate que detecta automáticamente qué atributos de una entidad gestionada han cambiado y genera solo el SQL necesario para actualizarlos al hacer commit. No hace falta llamar explícitamente a `update()`.

6. Patrón DAO

El **patrón DAO** (Data Access Object) encapsula todas las operaciones de persistencia de una entidad en una sola clase, separando el acceso a datos de la lógica de negocio.

JAVA — ProducteDAO con operaciones CRUD

```
public class ProducteDAO {
    private final SessionFactory sf;

    public ProducteDAO(SessionFactory sf) { this.sf = sf; }

    // Guardar
    public Long save(Producte p) {
        try (Session s = sf.openSession()) {
            Transaction tx = s.beginTransaction();
            Long id = (Long) s.save(p); tx.commit(); return id;
        }
    }

    // Buscar por ID
    public Producte getById(Long id) {
        try (Session s = sf.openSession()) {
            return s.get(Producte.class, id);
        }
    }

    // Obtener todos
    public List<Producte> getAll() {
        try (Session s = sf.openSession()) {
            return s.createQuery("from Producte", Producte.class).list();
        }
    }

    // Actualizar
    public void update(Producte p) {
        try (Session s = sf.openSession()) {
            Transaction tx = s.beginTransaction();
            s.update(p); tx.commit();
        }
    }

    // Eliminar
    public void delete(Producte p) {
        try (Session s = sf.openSession()) {
            Transaction tx = s.beginTransaction();
            s.delete(p); tx.commit();
        }
    }
}
```

✓ Ventajas del patrón DAO

La lógica de acceso a datos queda en un solo lugar — fácil de mantener y probar.
La capa de servicio solo maneja lógica de negocio, sin SQL ni sesiones.
Si cambias de base de datos o de ORM, solo tocas el DAO.

Bloque 8.2 · Proyecto completo: Hibernate + PostgreSQL

1. Software necesario

Para el proyecto necesitas tener instalado y configurado el siguiente software:

Herramienta	Versión	Para qué sirve
Java JDK	21 (Temurin)	Entorno de ejecución Java
PostgreSQL	16	Base de datos relacional
Visual Studio Code	Última estable	Editor + extensiones Java/Maven
Apache Maven	Incluido en VS Code	Gestor de dependencias y build

💡 Verificar la instalación

Tras instalar cada herramienta, verifica con: `java -version`, `psql --version`, `mvn -version`.

En macOS puedes instalar todo con Homebrew: `brew install temurin21 postgresql@16 maven`

2. Configurar PostgreSQL

Una vez instalado PostgreSQL, crea el usuario y la base de datos del proyecto desde la consola `psql` con el usuario administrador `postgres`:

SQL — Crear usuario y base de datos

```
-- Conectar como superusuario
psql -U postgres

-- Crear usuario dedicado al proyecto
CREATE USER hibuser WITH PASSWORD 'password';

-- Crear la base de datos
CREATE DATABASE hibernatedb OWNER hibuser;
GRANT ALL PRIVILEGES ON DATABASE hibernatedb TO hibuser;

-- Dar permisos sobre el esquema
\c hibernatedb
GRANT ALL ON SCHEMA public TO hibuser;
```

Verifica que el usuario puede conectarse:

```
psql -h localhost -U hibuser -d hibernatedb -W
# Si ves el prompt hibernatedb=> la configuración está lista ✓
```

⚠ Error en Linux: «Peer authentication failed»

Edita `/etc/postgresql/XX/main/pg_hba.conf` y cambia los métodos `peer` por `md5` en las conexiones locales. Después reinicia: `sudo systemctl restart postgresql`.

3. Estructura del proyecto Maven

Crea el proyecto desde VS Code: `Ctrl+Shift+P` → Maven: Create Maven Project → maven-archetype-quickstart. Usa `cat.exemple` como Group Id y `hibernate-demo` como Artifact Id.

```
hibernate-demo/
├── pom.xml
├── src/main/java/cat/exemple/
│   ├── util/      HibernateUtil.java
│   ├── model/     Producte.java
│   ├── dao/       ProducteDAO.java
│   └── Application.java
└── src/main/resources/
    └── hibernate.cfg.xml
```

4. pom.xml — Dependencias Maven

```
<properties>
  <hibernate.version>6.4.4.Final</hibernate.version>
  <postgresql.version>42.7.2</postgresql.version>
  <maven.compiler.source>21</maven.compiler.source>
</properties>

<dependencies>
  <!-- Hibernate Core -->
  <dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>${hibernate.version}</version>
  </dependency>

  <!-- Driver JDBC PostgreSQL -->
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>${postgresql.version}</version>
  </dependency>

  <!-- Jakarta Persistence API -->
  <dependency>
```

```

        <groupId>jakarta.persistence</groupId>
        <artifactId>jakarta.persistence-api</artifactId>
        <version>3.1.0</version>
    </dependency>
</dependencies>

```

5. hibernate.cfg.xml — Configuración

Crea el fichero `src/main/resources/hibernate.cfg.xml`:

XML — hibernate.cfg.xml

```

<hibernate-configuration>
  <session-factory>
    <property name="hibernate.connection.driver_class">
      org.postgresql.Driver
    </property>
    <property name="hibernate.connection.url">
      jdbc:postgresql://localhost:5432/hibernatedb
    </property>
    <property name="hibernate.connection.username">hibuser</property>
    <property name="hibernate.connection.password">password</property>
    <property name="hibernate.dialect">
      org.hibernate.dialect.PostgreSQLDialect
    </property>
    <property name="hibernate.hbm2ddl.auto">update</property>
    <property name="hibernate.show_sql">true</property>
    <property name="hibernate.format_sql">true</property>

    <mapping class="cat.exemple.model.Producte" />
  </session-factory>
</hibernate-configuration>

```

⚠ hbm2ddl.auto = update

En desarrollo **update** crea o modifica las tablas automáticamente al arrancar. En producción usa herramientas de migración como Flyway o Liquibase.

6. Implementación de las clases

6.1 HibernateUtil — Singleton de SessionFactory

JAVA — HibernateUtil.java

```

package cat.exemple.util;

public class HibernateUtil {
    private static final SessionFactory sessionFactory;

    static {
        try {
            Configuration cfg = new Configuration();
            cfg.configure("hibernate.cfg.xml");
            cfg.addAnnotatedClass(Producte.class);
            ServiceRegistry sr = new StandardServiceRegistryBuilder()
                .applySettings(cfg.getProperties()).build();

```

```

        sessionFactory = cfg.buildSessionFactory(sr);
    } catch (Throwable ex) {
        throw new ExceptionInInitializerError(ex);
    }
}

public static SessionFactory getSessionFactory() { return sessionFactory; }

public static void shutdown() {
    if (sessionFactory != null && !sessionFactory.isClosed())
        sessionFactory.close();
}
}

```

6.2 Producte — Entidad JPA

JAVA — Producte.java

```

package cat.exemple.model;

@Entity @Table(name = "productes")
public class Producte {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "nom", nullable = false, length = 100)
    private String nom;

    @Column(name = "preu", precision = 10, scale = 2)
    private BigDecimal preu;

    @Column(name = "stock")
    private Integer stock;

    public Producte() {} // constructor vacío obligatorio en JPA

    public Producte(String nom, BigDecimal preu, Integer stock) {
        this.nom = nom; this.preu = preu; this.stock = stock;
    }

    // getters, setters y toString()...
}

```

6.3 Application — CRUD completo

JAVA — Application.java (método main)

```

public static void main(String[] args) {
    try {
        crearProductes(); // INSERT
        mostrarTots();    // SELECT *
        actualitzarProducte(); // UPDATE
        cercarPerNom("portàtil"); // SELECT WHERE
        eliminarProducte(); // DELETE
        mostrarTots();    // SELECT * final
    } finally {
        HibernateUtil.shutdown();
    }
}

```

7. Compilar y ejecutar

BASH — Comandos Maven

```
mvn clean compile # compilar
mvn test          # ejecutar tests JUnit
mvn exec:java -Dexec.mainClass="cat.exemple.Application" # ejecutar
```

Tras ejecutar, verifica el resultado directamente en PostgreSQL:

SQL — Verificar en PostgreSQL

```
psql -h localhost -U hibuser -d hibernatedb -W
\dt # listar tablas
SELECT * FROM productes; # ver registros
\q
```

Problema	Solución
Peer authentication failed (Linux)	Cambiar método en pg_hba.conf a md5 + reiniciar PostgreSQL
Error de conexión	Verificar URL, usuario y contraseña en hibernate.cfg.xml
BUILD FAILURE en Maven	Ejecutar Maven: Clean y Maven: Update project en VS Code
Tablas no se crean	Comprobar que hbm2ddl.auto=update y la entidad está mapeada

8. Relaciones entre entidades

Hibernate gestiona las relaciones entre tablas mediante anotaciones. El ejemplo más común es ManyToOne / OneToMany (ej: una Categoría tiene muchos Productos).

JAVA — Relación Categoría → Producto

```
// En Categoría.java
@OneToMany(mappedBy = "categoria", cascade = CascadeType.ALL)
private List<Producto> productes = new ArrayList<>();

// En Producto.java - lado propietario de la FK
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "categoria_id")
private Categoria categoria;
```

Relación	Anotación	Ejemplo
Uno a muchos	@OneToMany / @ManyToOne	Categoría → Productos
Muchos a muchos	@ManyToMany	Estudiantes ↔ Asignaturas
Uno a uno	@OneToOne	Persona ↔ Pasaporte

Ejercicios de repaso · Unidad 8

Ejercicio 1 — Anotar una entidad (Fácil · 10 min)

Anota la clase `Client` para que sea persistente con JPA. La clave primaria debe ser autogenerada y el atributo `email` no puede ser nulo ni repetido.

- Añade `@Entity` y `@Table(name = "clients")` a la clase.
- Marca el atributo `id` con `@Id` y `@GeneratedValue(strategy = GenerationType.IDENTITY)`.
- Añade `@Column(nullable = false, length = 80)` al atributo `nom`.
- Añade `@Column(nullable = false, length = 120, unique = true)` al atributo `email`.
- Registra la clase en `HibernateUtil` y en `hibernate.cfg.xml`.

Ejercicio 2 — Refactorizar JDBC a Hibernate (Medio · 15 min)

Reescribe este fragmento JDBC usando Hibernate y el patrón DAO: `String sql = "UPDATE productos SET preu = ? WHERE id = ?";`

- Crea un método `actualizarPrecio(Long id, BigDecimal nuevoPrecio)` en el DAO.
- Abre una sesión con `sf.openSession()` dentro de un try-with-resources.
- Inicia una transacción con `session.beginTransaction()`.
- Carga la entidad con `session.get(Producto.class, id)`.
- Modifica el precio con `prod.setPreu(nuevoPrecio)` y haz `tx.commit()`.
- En el catch, llama a `tx.rollback()` si tx no es null.

Ejercicio 3 — CRUD completo (Medio · 25 min)

Implementa el proyecto hibernate-demo completo y ejecuta las operaciones CRUD verificando el resultado en PostgreSQL.

- Crea el proyecto Maven con la estructura indicada.
- Configura `pom.xml` con las dependencias de Hibernate 6 y PostgreSQL.
- Crea `hibernate.cfg.xml` con los datos de conexión a hibernatedb.
- Implementa `HibernateUtil`, `Producto`, `ProductoDAO` y `Application`.
- Ejecuta con `mvn exec:java` y verifica con `SELECT * FROM productos;` en psql.

Ejercicio 4 — Añadir entidad Categoría (Avanzado · 25 min)

Amplía el proyecto añadiendo la entidad `Categoría` con relación OneToMany → `Producto`.

- Crea la clase `Categoría` con `@Entity`, `@Id` y `@OneToMany(mappedBy = "categoria")`.
- Añade `@ManyToOne` y `@JoinColumn(name = "categoria_id")` en `Producto`.
- Registra `Categoría` en `HibernateUtil` y en `hibernate.cfg.xml`.
- Crea un `CategoríaDAO` con los métodos `save` y `getAll`.
- En `Application`, crea categorías, asígnalas a productos y muéstralas al lista.